Ca' Foscari University of Venice

Development Methodology

Matteo Zanioli

matteo.zanioli@unive.it

Ca' Foscari University of Venice

Which kind of tests do you know?

1 Go to wooclap.com

2 Enter the event code in the top banner

Event code
DAECRJ

Kind of tests

- Unit testing: test individual components of functions in isolation
- Integration testing: verifies that different modules or services work together correctly
- Functional testing: ensure that the software meets specified functional requirements
- End-to-End (E2E) testing: test the entire application flow from start to finish
- Acceptance testing: test which determines if the system satisfies the acceptance criteria and is ready for production
- Performance testing: evaluates the system's performance under various conditions

Kind of tests

- Load testing: assesses how the system performs under expected load conditions
- Stress testing: examines system behavior under extreme conditions or beyond normal operational capacity
- Security testing: identifies vulnerabilities and ensures the system is protected against various types of attacks
- Usability testing: evaluates the user interface and overall user experience
- Smoke testing: quick test to check if the basic functionality works before more thorough testing
- much more... (regression, not functional, etc)

Definition of testing

- Testing is a **systematic process** of **evaluating** a product, system, or component to **detect differences between** expected and actual results, **verify** compliance with specified requirements and **identify potential defects** or areas for **improvement**.

- Categorization: static and dynamic

Static vs dynamic testing

Static

- Performed without executing the code
- Involves reviewing documents, code, and other artifacts
- Can be done early in the development process
- Examples include:
  - code reviews
  - static code analysis
  - Design and architecture reviews
  - Documentation reviews

Dynamic

- Involves executing the code
- Observes the behavior of the system under test
- Usually performed later in the development process
- Examples include:
  - Unit testing
  - Integration testing
  - System testing
  - Performance testing

Static vs dynamic testing

- Both type of testing are valuable and complementary in the software development process.
- Static testing can catch issues early and improve code quality
- Dyanamic testing verifies the actual behavior of the running system

Ca' Foscari University of Venice

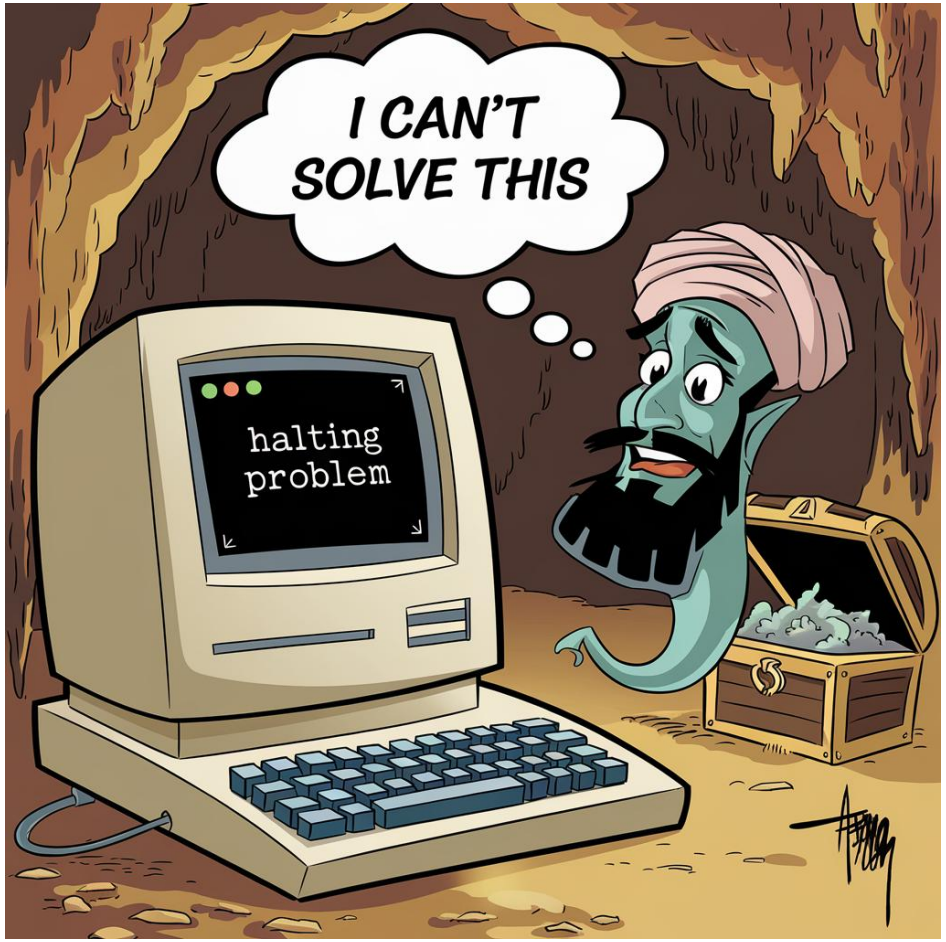
Is it possible to check everything in software through testing?

1. Go to wooclap.com
2. Enter the event code in the top banner

Event code
KFHRJS

Theory: halting problem

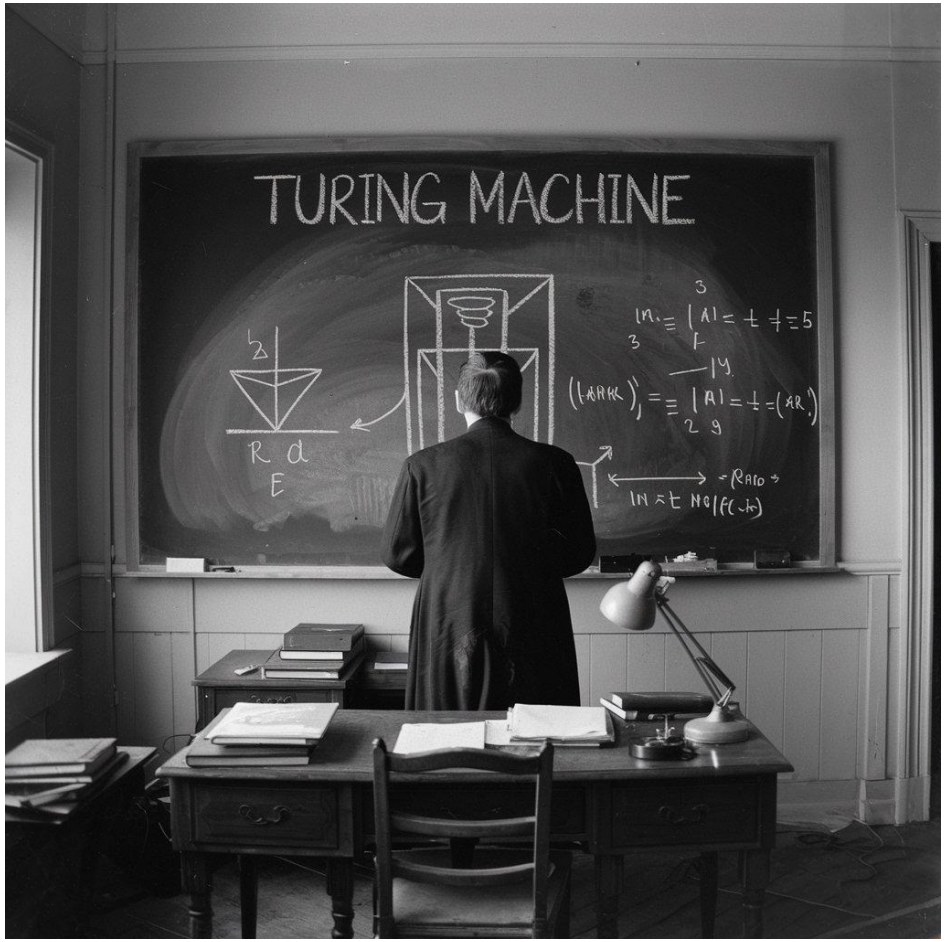
Do you know it?



Ca' Foscari University of Venice

Halting problem

- Do you know Alan Turing? Contributions of Alan Turing:
  - The Turing Machine – theoretical computing model and concept of computation
  - Breaking the Enigma Code – World War II
  - Turing Test – Machine intelligent behavior
  - ... (many more)
  - **Halting Problem**



Halting problem

- Question: is it possible to create a program that can determine, for every possible program and input, whether that program will eventually halt (terminate) or run forever?
- Answer: "On computable numbers, with an application to the Entscheidungsproblem, A. M. Turing, 1936 "https://londmathsoc.onlinelibrary.wiley.com/doi/abs/10.1112/plms/s2-42.1.230
- No, it is not possible. This program does not exist.

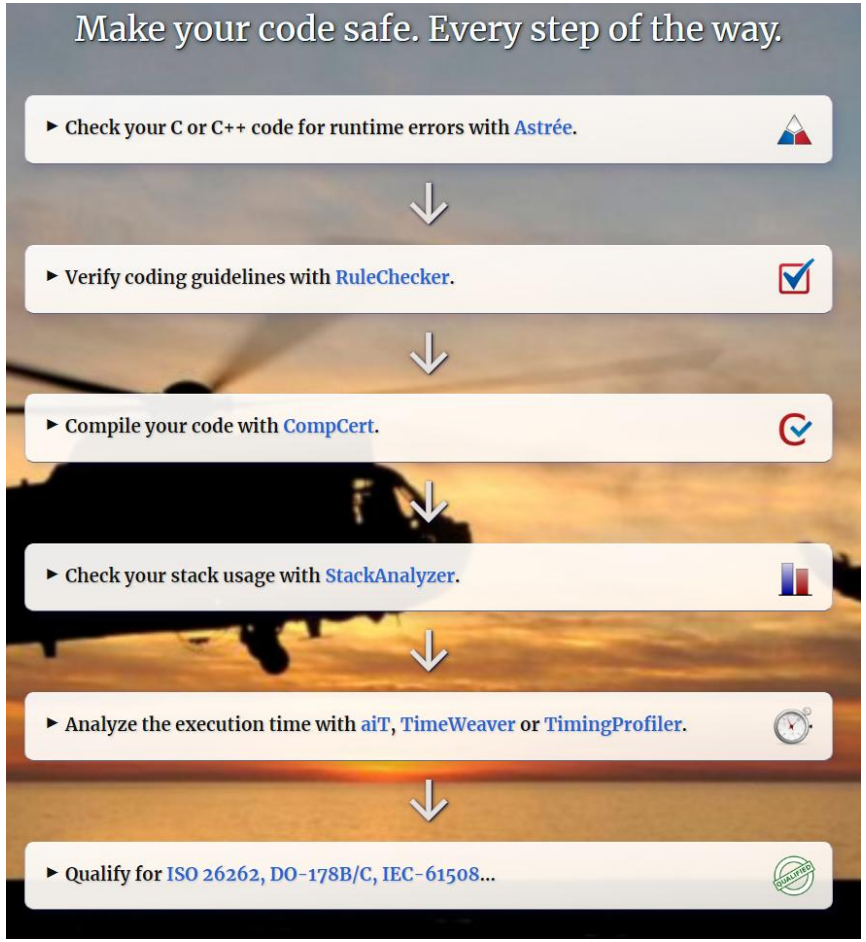
Implication with testing?

- Halting problem demonstrates that there are fundamental limitations to what can be achieved through testing and analysis of software
- It's impossible to create a general testing method that can detect all possible bugs or issues in a program
- Automatic test generation faces limitations: it's not always possible to automatically determine if a generated test case will terminate
- Heuristics and approximations are often used to provide useful results in static analysis and testing tools
- Many testing and analysis techniques work around halting problem limitations by imposing bounds (e.g., time limits, depth limits in static analysis)

Ca' Foscari University of Venice

Static testing

- Abstract Interpretation
  - approximates the semantics of the program to infer properties about its runtime behavior
  - https://www.absint.com/index.htm with their tool
  - https://pcousot.github.io/talks/Cousot-ISOLA07_1-1.pdf



AIRBUS BOSCH esa

Static testing

- Type checking
  - Verifies type consistency in the code
  - Can be done by compilers or separate tools
  - Important in statically-typed languages
  - examples: TypeScript's type checker, Rust's borrow checker
- Formal verification
  - Uses mathematical methods to prove or disprove the correctness of algorithms
  - Techniques include model checking and theorem proving
  - examples: Coq (https://coq.inria.fr/), Isabelle (https://isabelle.in.tum.de/), TLA+ (https://github.com/tlaplus)

Static testing

- Linting
  - Analyzes code for stylistic errors, potential bugs, and suspicious constructs
  - examples: ESLint (https://eslint.org/), Pylint (https://pypi.org/project/pylint/), RuboCop (https://rubocop.org/)
- Software Metrics Analysis
  - Measures various properties of the software like complexity, cohesion, coupling
  - examples: SonarQube (https://www.sonarsource.com/products/sonarqube/), CodeClimate (https://codeclimate.com/)

Dynamic analysis

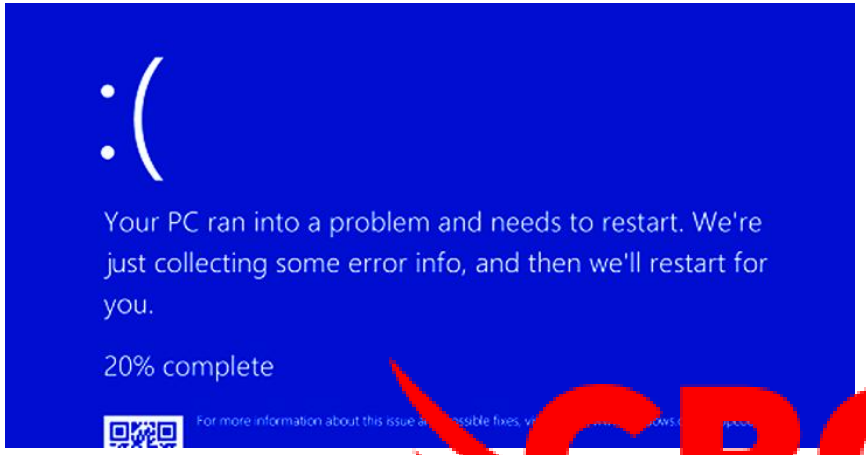
- Java Unit Testing – JUnit - https://junit.org/
- Automation Testing for Web Applications – Selenium - https://www.selenium.dev/
- PyTest - https://docs.pytest.org/
- Performance test - Apache JMeter - https://jmeter.apache.org/
- Functional and regression testing – Katalon Studio - https://katalon.com/
- Automating mobile application test tool – Appium - https://appium.io/
- API testing – Postman - https://www.postman.com/
- Behavior-driven development (BDD) – Cucumber - https://cucumber.io/
- and many more...

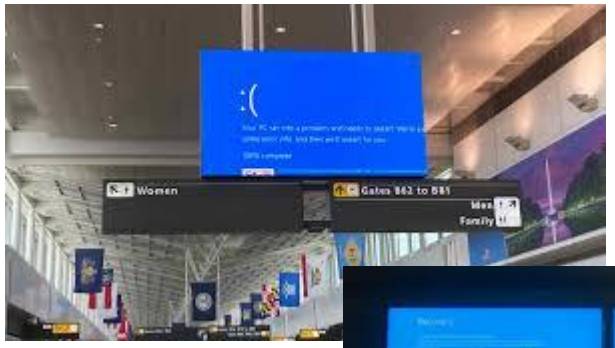
Question

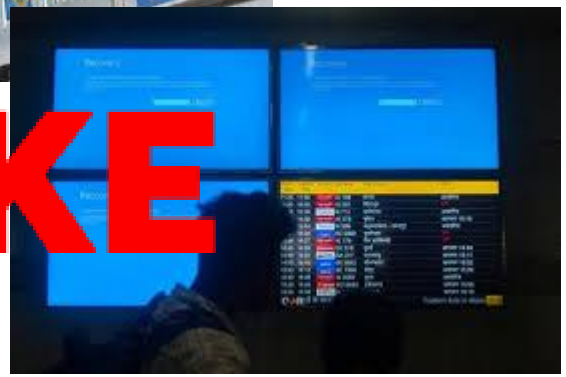
Is it really important to perform tests?

Ca' Foscari University of Venice

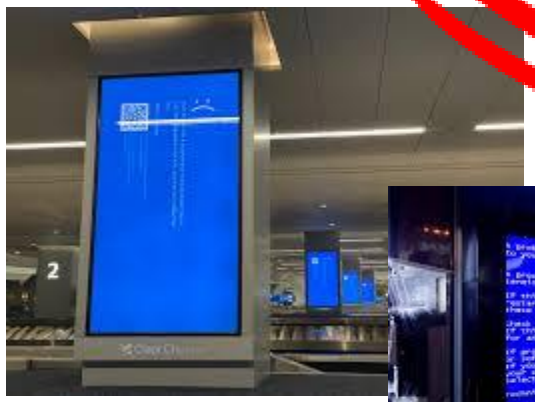
Real case: why tests are so important?







CROWDSTRIKE



What might be considered the largest IT outage in history was triggered by a botched software update from security vendor CrowdStrike, affecting millions of Windows systems around the world. Insurers estimate the outage will cost U.S. Fortune 500 companies $5.4 billion.

The outage occurred July 19, 2024, with millions of Windows systems failing and showing the infamous blue screen of death (BSOD).

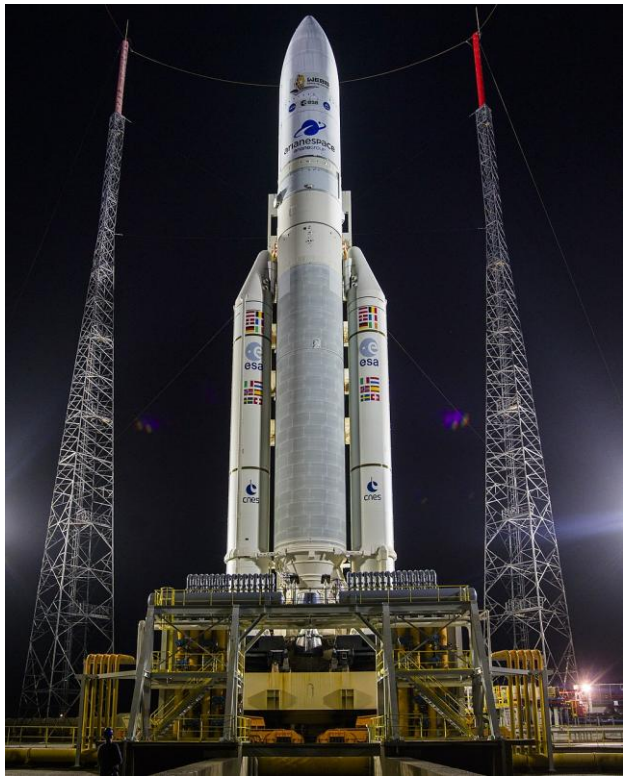
In terms of how the logic error was introduced into the Falcon sensor, the issue was a failure in the CrowdStrike development process. On July 24, 2024, the company issued a preliminary Post Incident Review (PIR). According to the PIR, there was a flaw in CrowdStrike's Content Validator component, used to check the integrity of rapid response content update. That flaw enabled the faulty version of channel file 291 to pass validation, even though it had an error.

source: https://tinyurl.com/2c8edx3z

Ca' Foscari University of Venice

Real case: why tests are so important?

The Worst Computer Bugs in History: The Ariane 5 Disaster



What went wrong?

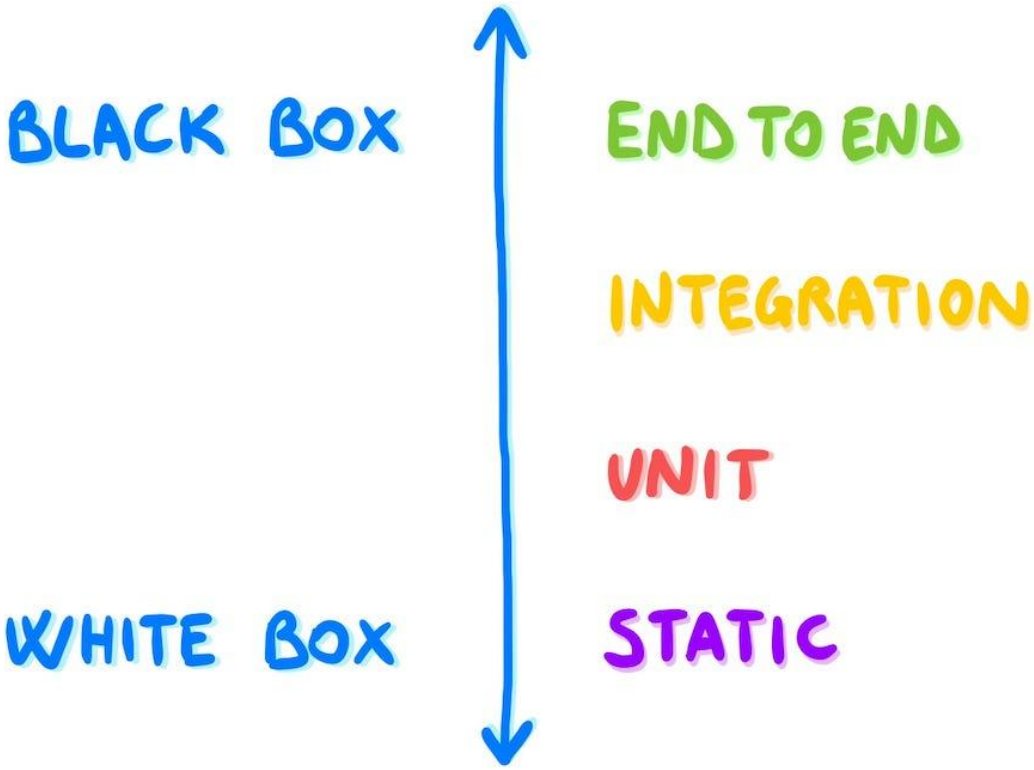
The fault was quickly identified as a software bug in the rocket's Inertial Reference System. The rocket used this system to determine whether it was pointing up or down, which is formally known as the horizontal bias, or informally as a BH value. This value was represented by a 64-bit floating variable, which was perfectly adequate.

However, problems began to occur when the software attempted to stuff this 64-bit variable, which can represent billions of potential values, into a 16-bit integer, which can only represent 65,535 potential values. For the first few seconds of flight, the rocket's acceleration was low, so the conversion between these two values was successful. However, as the rocket's velocity increased, the 64-bit variable exceeded 65k, and became too large to fit in a 16-bit variable. It was at this point that the processor encountered an operand error, and populated the BH variable with a diagnostic value.

sources: https://www.bugsnag.com/blog/bug-day-ariane-5-disaster/
sources: https://en.wikipedia.org/wiki/Ariane_5

Tests in practice

- If you imagine tests as a pyramid with unit tests at the bottom and end-to-end tests at the top, the higher you go, the more complex the workflows you're testing, and the darker this box becomes because you can ignore more layers below



source: https://substack.com/home/post/p-148995241

Tests in practice

- All types of tests verify that some contract is respected, while treating the implementation as a black box
- The various types of tests, such as unit, integration, or end-to-end, differ mostly in the scope and size of such contracts and black boxes
- For example, the way a UI works when you click things around is a contract between the software and the user. Similarly, the signature and semantics of a function represent a contract between the function and the other code that invokes it
- Tests avoid regressions by enforcing these contracts whenever you make changes to their underlying implementations

source: https://substack.com/home/post/p-148995241

Tests in practice

- Based on how and when the contract or the implementation changes, you have three scenarios
  - contract does not change and implementation does not change: test is not useful
  - contract does not change and implementation changes: test can catch a regression
  - contract changes and implementation changes: you need to change the test
- The only scenario in which a test repays itself is the second one. In the first one, the test is irrelevant, while in the last one, it is a liability because you have to update it to reflect the new contract.

source: https://substack.com/home/post/p-148995241

Tests in practice

	DEV EFFORT	CHANGE CONTRACT	SWAP IMPLEMENTATION	ROI
UNIT TEST	LOW	OFTEN	RARELY	MEDIUM
INTEGRATION TEST	MEDIUM	RARELY	OFTEN	HIGH
END TO END TEST	HIGH	OFTEN	OFTEN	MEDIUM

- End-to-end tests are not particularly hard to write, but
  - Speed: End-to-end tests need browsers / device simulators, which makes them slow.
  - Reliability: End-to-end tests generate a lot of false positives, also known as flakes
- higher the level of the test, the more things you have below that can fail. With end-to-end, this includes anything from network, to external API calls, browser quirks, and infrastructure issues.

source: https://substack.com/home/post/p-148995241

Ca' Foscari University of Venice

Tests in practice

- So, every time a test fails, you have to figure out whether:
  - It is just a flaky test and a re-run fixes it, or
  - The product is fine but the test needs to be updated, or
  - It is an actual bug
- Only in the third scenario the test is useful — in the other ones it is a liability

- All of this makes end-to-end tests high maintenance, which is what discourages many teams
- the time spent investigating tests is typically higher than doing manual QA

source: https://substack.com/home/post/p-148995241

Ca' Foscari University of Venice

Real case: unit test for JS

- Unit test on functions
- Using specific library depends on specific languages/framework (e.g., React Testing Library and JEST – Javascript testing framework)
- Implementation within the "merge-pipeline": the code will be merged if and only if all unit tests passed

229	229 Passed	100%	2m 47s	0
Total tests	0 Failed	Pass percentage	Run duration	Tests not reported
	0 Others		53s 430ms	

54.18%	151	3601	1951
Line Coverage	Modules	Total Lines	Covered Lines

```
PASS  src/overrides/app/components/lazy-cdn-script/index.test.js
  Lazy CDN Script Test
    √ should render nothing (11 ms)
    √ should not load the script when it already exists in the DOM (3 ms)

PASS  src/overrides/app/components/loading-spinner/index.test.js
  LoadingSpinner
    √ should render (51 ms)

PASS  src/overrides/app/components/progress-bar/index.test.js
  Progress Bar Component Test
    √ should render progress bar (33 ms)
```

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	55.6	37.45	40.56	56.23	
components/_error	100	100	100	100	
index.jsx	100	100	100	100	
components/breadcrumb	100	100	100	100	
index.jsx	100	100	100	100	
components/collapsible-chips	100	93.33	100	100	
index.jsx	100	93.33	100	100	12
components/conditional-rendering	100	100	100	100	
PreferenceBasedVisibility.jsx	100	100	100	100	
ShowIfIsEcommerce.jsx	100	100	100	100	

Ca' Foscari University of Venice

Real case: SonarQube

"calc" operands should be correctly spaced — CSS Bug

"at-rules" should be valid

"!important" should not be used on "keyframes"

"Bean Validation" (JSR 380) should be properly configured

"compareTo" should not be overloaded

"action" mappings should not have too many "forward" entries — Java Code S

"compareTo" should not return "Integer.MIN_VALUE"

"close()" calls should not be redundant — Java

"@Import"s should be preferred to "@ComponentScan"s — Java Code

"@EnableAutoConfiguration" should be fine-tuned — Java Code

"@RequestMapping" methods should be "public" — Java Vul

"@Controller" classes that use "@SessionAttributes" must call "setComplete" on their "SessionStatus" objects

"Arrays.stream" should be used for primitive arrays — Java

sonarqube | Projects | Issues | Rules | Quality Profiles | Quality Gates | More | Search for projects... | MZ

My Favorites | All

Filters

Quality Gate
- Passed 51
- Failed 7

Reliability (Bugs)
- A 33
- B 0
- C 8
- D 4
- E 13

Security (Vulnerabilities)
- A 54
- B 0
- C 0
- D 2
- E 2

Security Review (Security Hotspots)
- A ≥ 80% 21
- B 70% - 80% 0
- C 50% - 70% 4
- D 30% - 50% 1
- E < 30% 32

Maintainability (Code Smells)
- A 58
- B 0
- C 0
- D 0
- E 0

Coverage
- ≥ 80% 1

Perspective: Overall Status | Sort by: Name | Search by project name or key | 66 projects

Project	Last analysis	Bugs	Vulnerabilities	Hotspots Reviewed	Code Smells	Coverage	Duplications	Lines	
alpenite-acciaieriebertolisafau-acc (Passed)	Last analysis: 8 months ago	0 A	0 A	0.0% E	0 A	0.0%	0.0%	7 XS	PHP
alpenite-acciaieriebertolisafau-mag2ee (Passed)	Last analysis: 8 hours ago	109 D	0 A	– A	144 A	0.0%	1.1%	8k S	CSS, PHP, ...
alpenite-artista-apis (Passed)	Last analysis: 2 years ago	0 A	0 A	– A	1 A	0.0%	0.0%	1.1k S	Go, HTML
alpenite-artista-cms (Passed)	Last analysis: 2 years ago	0 A	0 A	– A	0 A	0.0%	0.0%	124 XS	JavaScript
alpenite-artista-core (Passed)	Last analysis: 2 years ago	0 A	0 A	– A	1 A	0.0%	0.8%	6.7k S	Go

PRIVATE

Ca' Foscari University of Venice

Real case: Cypress and cucumber

- Pseudocode language
- Tests similarly to the natural language
- GWT paradigm
- Business can understand tests for formal approval

cypress cucumber

- GWT paradigm (Given, When, Then) example

```
Feature: Logging In

Test: tests the user can successfully login
and log out

Scenario: Logging In Sucessfully
    Given I am on the "login" page
    When I input my "email" as test@test.com
    And I input my "password" as "password"
    And I click the "login" button
    Then I should be on the "dashboard" page
```

Ca' Foscari University of Venice

Real case: Cypress and cucumber

A	B	C	D	E	F	G	H	I	J	K
Id	Resolution	Macro area	Specific area	Test	Automatic/manual	Writer	priorità	state	note	implemented
1	all	login	page view	Given as a user When I'm browsing the login page Then I see two areas: "Registered User" and "New User" And in "Registered User" area there're email and password form, title, flag remember me, CTA Login and "forgot your password" link And in "new user" Area there're title, description, CTA "Create account", link "more details"		Marco Napetti		approved		loginPage/loginUser.feature
2	all	login	page view	Givenas a business user When I modify the content asset "New User" Then I see the new layout on the login page	manual	Marco Napetti				
3	all	login	page view	Given as a user When I'm browsing the Login page And I insert a wrong email Then an error message is displayed		Marco Napetti		approved		loginPage/loginUser.feature
4	all	login	page view	Given as a user When I'm browsing the Login page And I insert a not registered email Then the error message is displayed		Marco Napetti		approved		loginPage/loginUser.feature
5	all	login	page view	Given as a user When I'm browsing the Login page And I insert a registered email And I insert a wrong password Then the error message is displayed		Marco Napetti		approved		loginPage/loginUser.feature
6	all	login	page view	Given as a user When I'm browsing the Login page And I insert a valid email And I click on "Login" Then an error message "Password is required" appears		Marco Napetti		approved		loginPage/loginUser.feature
7	all	login	page view	Given as a user When I'm browsing the Login page And I insert a password And I click on "Login" Then an error message "Email is required" appears		Marco Napetti		approved		loginPage/loginUser.feature
				Given as a user When I'm browsing the Login page						

Exploration cases | Global Navigation | **Account management** | Product management | Cart & Checkout | Transactional Email

Ca' Foscari University of Venice

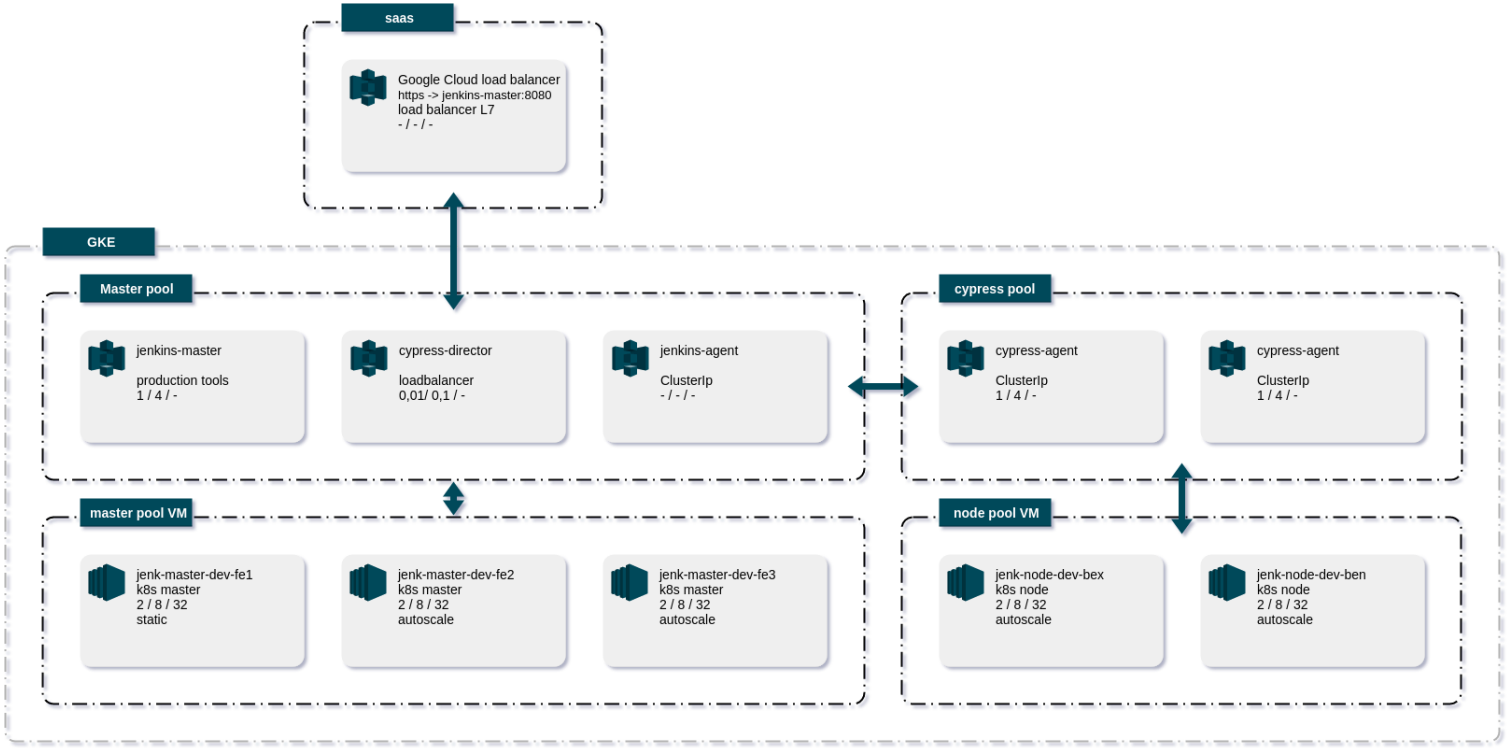
Real case: Cypress and cucumber

CI/CD engine

- Extensible with new functionalities
  - release on different platform
  - source code static analysis
- Multi-vendor support
- Implementable on customer hosting

Components

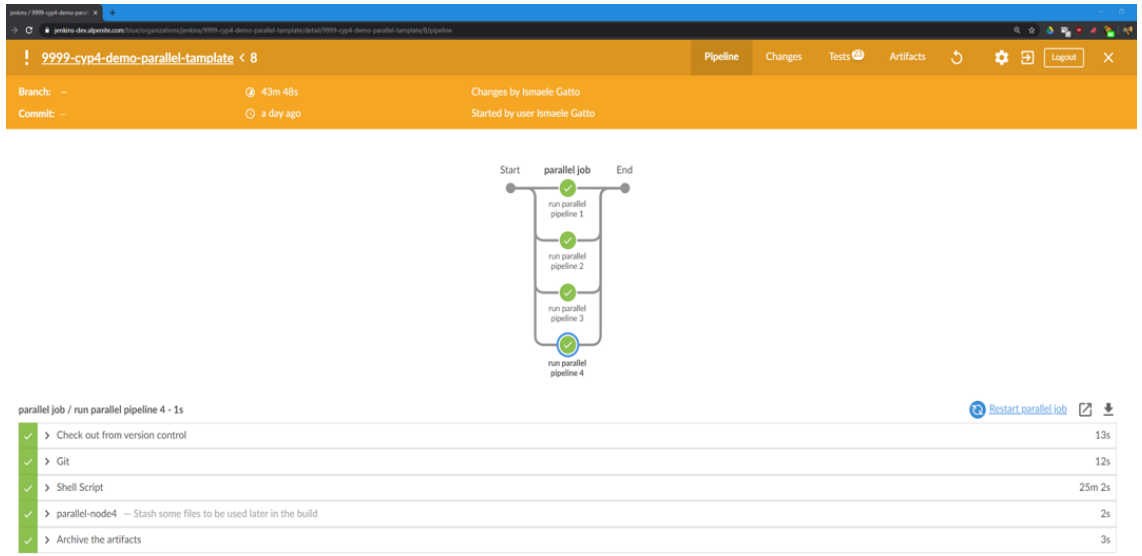
- Google Cloud load balancer
- Google Kubernetes engine
- Jenkins master
  - Jenkins-agent
- Cypress-agent
  - Jenkins-agent + node
- Cypress director
- Cluster reachable only by whitelisted IPs



Ca' Foscari University of Venice

Real case: Cypress and cucumber

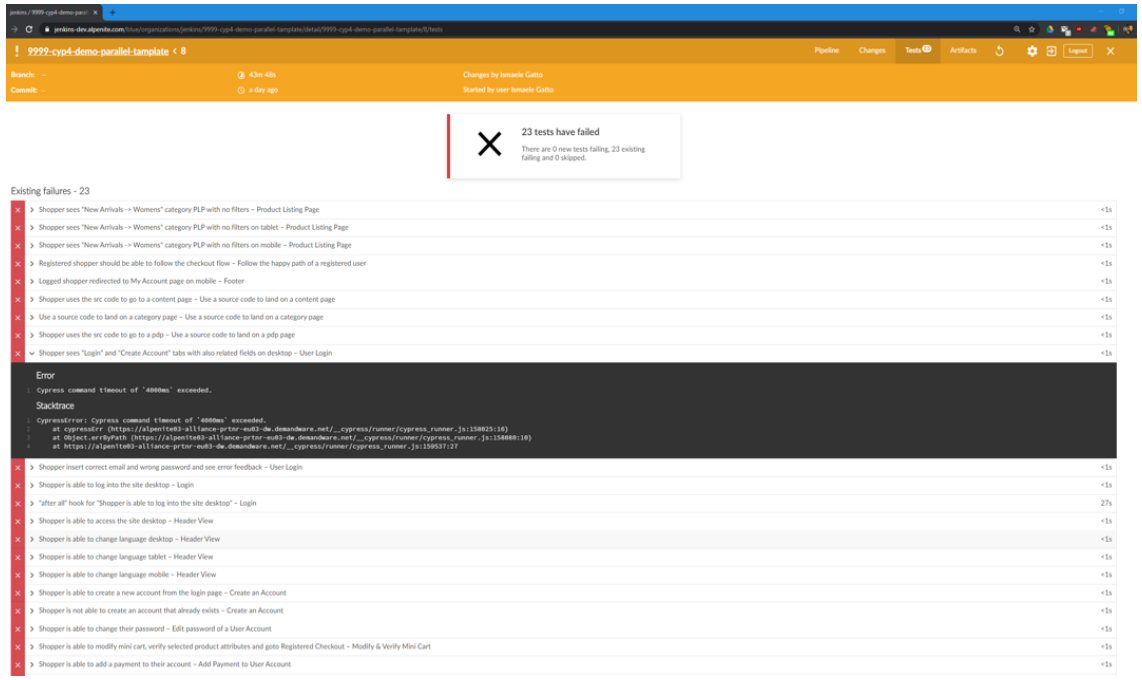
- Specific user agent
- Input from user and configuration
- Jobs parallelization
- For each job a new pod is created
- Caching and cypress installation
- Storing artifact and results



Ca' Foscari University of Venice

Real case: Cypress and cucumber

- Executed test browsing
- Stack traces of errors
- Artifacts generated by errors:
  - screenshot
  - video
- Artifacts download



Syntetic monitor

- Synthetic monitoring, also known as **active monitoring**, is a method used to monitor software applications or websites by **simulating user interactions**. **Instead of waiting for real users** to encounter issues, synthetic monitoring involves running automated scripts that mimic user behaviors to test the performance, availability, and functionality of an application in a controlled manner.

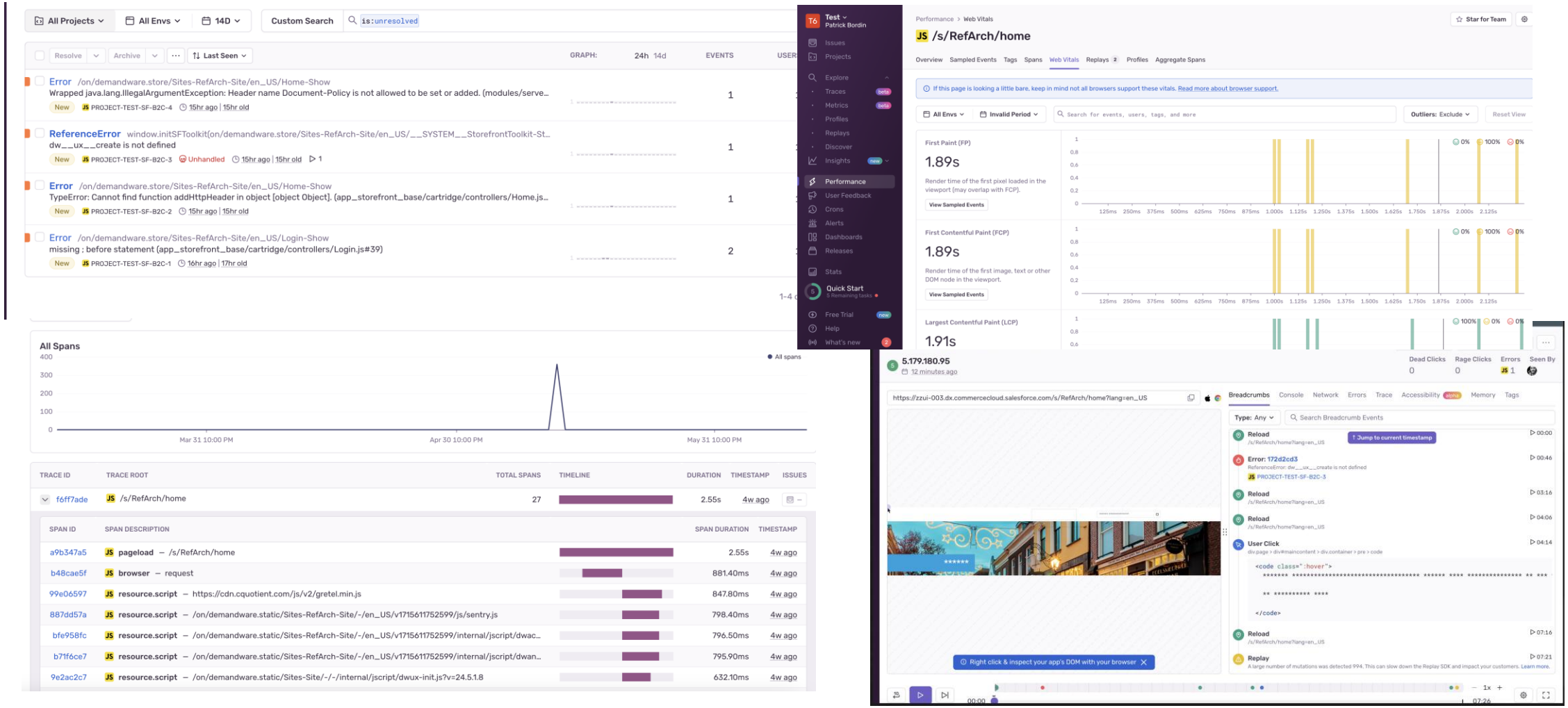
Syntetic monitor

- Simulated User Behavior: It involves creating scripts that perform specific actions, like logging in, navigating through pages, or completing transactions, to replicate how a real user would interact with the application.
- Proactive Testing: Since synthetic monitoring is continuous and automated, it allows companies to detect potential issues (like slow load times or downtime) before actual users are affected.
- Global Coverage: Synthetic monitors can be run from multiple locations around the world, providing insights into performance across different regions.

- Measuring Key Metrics:
  - Response time: Measures how long it takes for the system to respond to a user action.
  - Uptime and Availability: Monitors whether the application or website is online and available for use.
  - Functionality: Checks if core features (e.g., login, checkout) are working properly.
- Use Cases:
  - Performance Monitoring: Measure application speed and responsiveness under different conditions.
  - SLA Compliance: Ensure that service-level agreements (SLAs) for uptime and performance are met.
  - Regression Testing: Validate that new updates or deployments do not break existing functionalities.

Ca' Foscari University of Venice

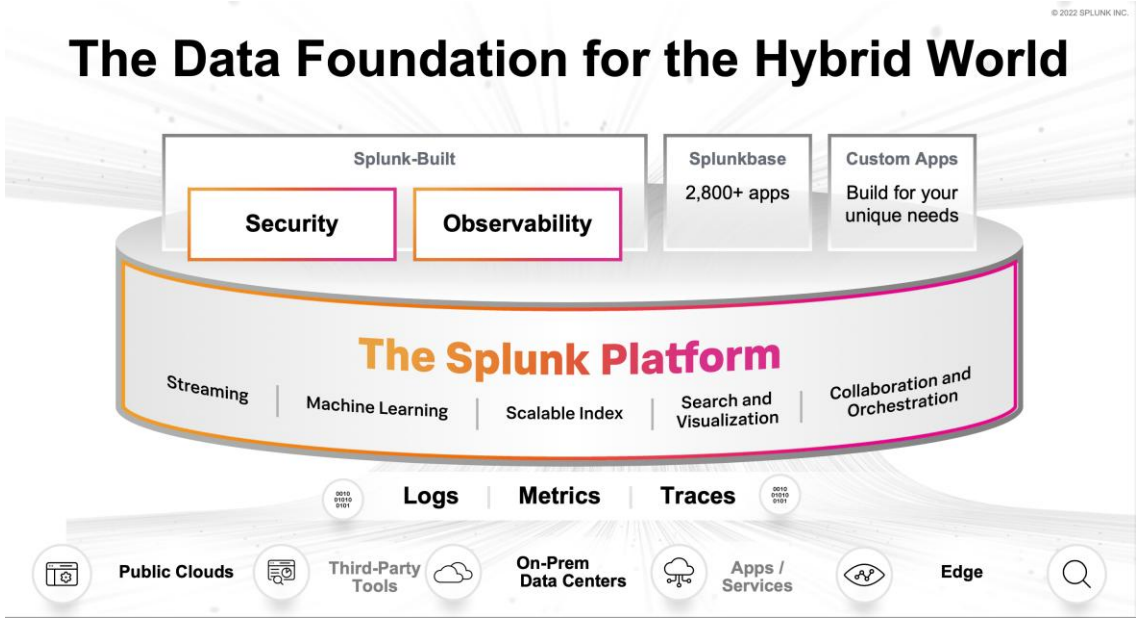
Monitoring frontend – sentry.io



Ca' Foscari University of Venice

Monitor tool

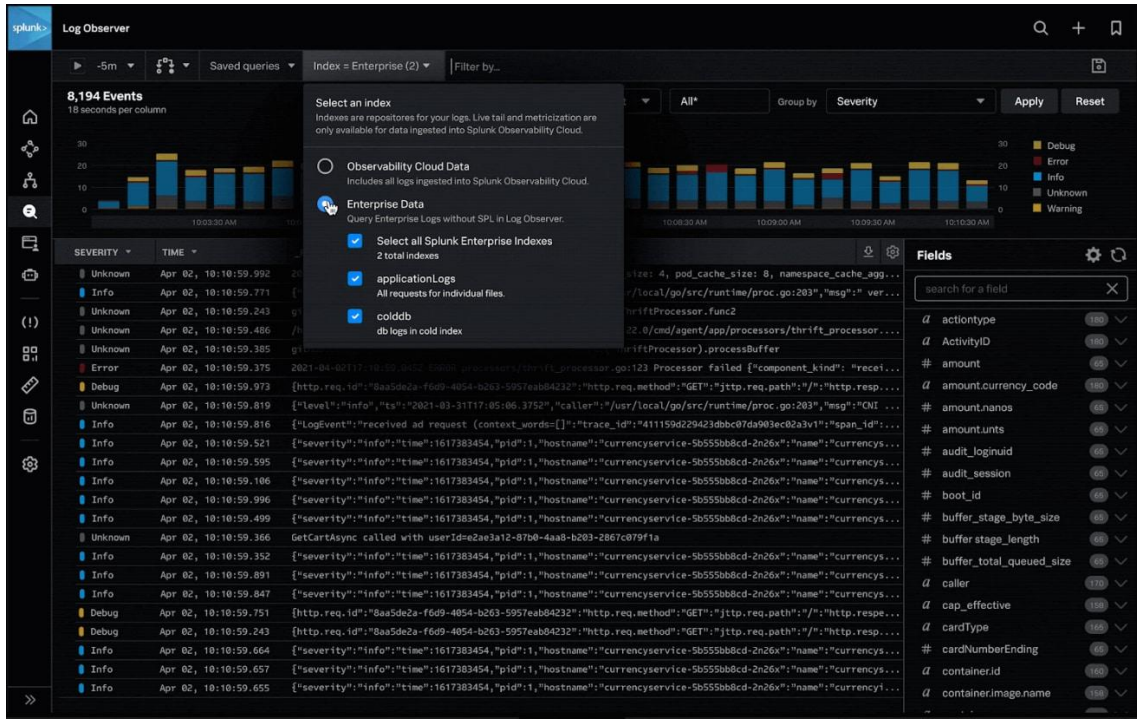
- Splunk is the observability platform
- Examples of modules:
  - Log Observer: monitoring of communication interfaces
  - Synthetic Monitoring: monitoring services from an external perspective



Ca' Foscari University of Venice

Monitor tool - observer

- Log Investigation for DevOps
- Fast start and fast troubleshooting
- Graphical Log Exploration without the need for a query language
- "Live Tail" - Real time log insights optimized for debugging
- Infinite Logging - Ingest all the data, index the most valuable data



Ca' Foscari University of Venice

Syntentic monitor

- The leading fully featured synthetic monitoring solution for application transactions
- Record and test user journeys in minutes
- Proactively reduce performance regression
- Emulate real-world service dependencies
- Robust, tightly integrated real-time notification
- Low touch performance baselining

Quality Assurance

- From OpenAI: *proactive process focus on optimizing the software development to ensure high-quality software. The goal is to optimize development and testing procedures to prevent problems from occurring. In essence, QA specialists uphold high standards and prevent defects at every stage of the software development lifecycle.*

QA is a mindset

- Actually, the defition above is not so satisfying. QA is a mindset, rather than a specific process.
- QA is making sure you meet an expectation of quality for your product, and in particular that you don't go below that quality
- This awareness about where the **value** is, is the reason why we talk about *Quality Assurance*, as a broad function, as opposed to simply *Testing*
- Good QA requires sorting out what's really valuable from what is not, and while you might argue this applies to testing in general, QA does this from a product/business standpoint, rather than a pure engineering one

source: https://substack.com/home/post/p-148995241

Quality Assurance

- In the early stages—pre-product market fit—your product changes rapidly, so tests easily become a liability rather than an asset. At this stage, you should invest very little in QA, especially automated QA
- Later, finding product-market-fit typically brings two things: 1) the stuff to test is more valuable than before, and 2) you can identify some parts of the product that look more stable. Together, these unlock more value from automated testing
- We already talk about the effort for end2end tests: The best teams minimize their need for end-to-end testing by guaranteeing high quality through the other stages of development – Quality Assurance

source: https://substack.com/home/post/p-148995241

Quality Assurance

- Great integration testing
  - Integration tests are easier to write and maintain than end-to-end
- Zero-defect policy
  - Foster a culture where bugs are fixed asap, even if low-priority. This is a no-brainer: having few-to-none known defects at any given time makes your testing more reliable and improves the morale of your team
- Good observability
  - Engineers should own the code they write even after it gets to prod. They should use good observability tools to verify that everything works as intended, and catch bugs early

source: https://substack.com/home/post/p-148995241

Quality Assurance

- Get engineers invested in quality
  - The strategies above may look easy or hard to you based on how much quality already belongs to your team's culture
  - Get buy-in about changing your culture
  - Expose developers to consequences
  - Make testing a first-class citizen of your process
  - If you are a manager, show your team that quality is important to you

source: https://substack.com/home/post/p-148995241

QA journey

- Zero-to-one → No Testing: Very little QA should be in place at this time. E2E would drag you down because the product changes too fast.
- Product-market-fit → Manual Testing: Customers start to rely on some workflows, and you have some consolidated features. Start documenting critical ones and doing manual QA on them. The ideal process looks like having a list of workflows you go through for each release.
- Scale → Automated testing: You have many of such workflows now, and manual testing has become a burden. It slows down releases, and people (engineers and PMs) despise it. Automate ~10 workflows yourself, create some playwright / cypress tests as part of your CI/CD, on an environment that is as close to prod as possible.
- Testing at scale: As you grow, when you start having 50-100 automated workflows, you enter an entirely different ball game, both from a tech and a team point of view

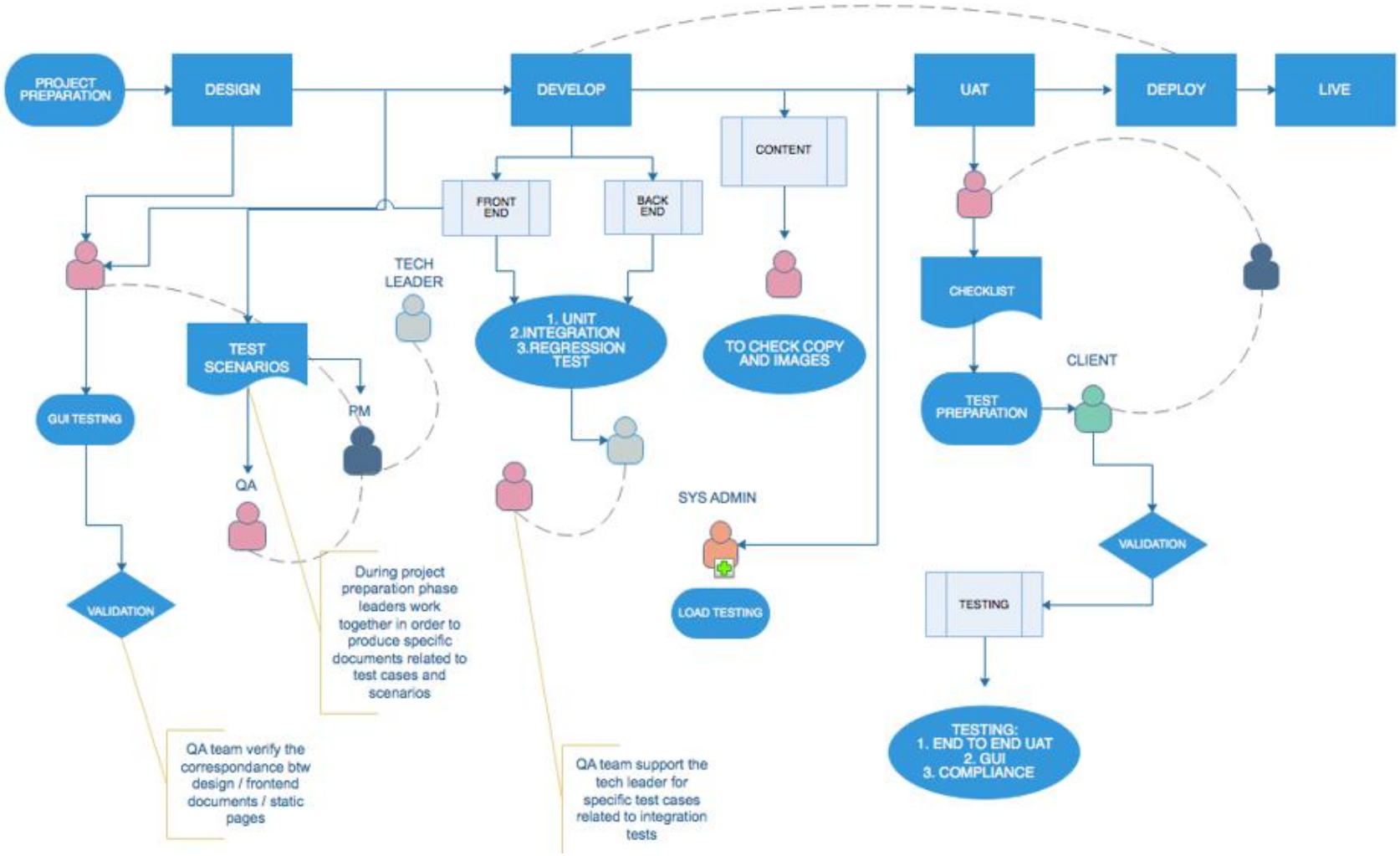
source: https://substack.com/home/post/p-148995241

Quality Assurance

- Quality should be owned by the product team: by the product engineers, and overseen by the product manager
- To run EtoE tests internally at scale, you may want to hire dedicated QA engineers
- End-to-end tests tend to be slow and resource-hungry, so creating the right environment for them makes a huge difference in making them fast and less prone to flakes

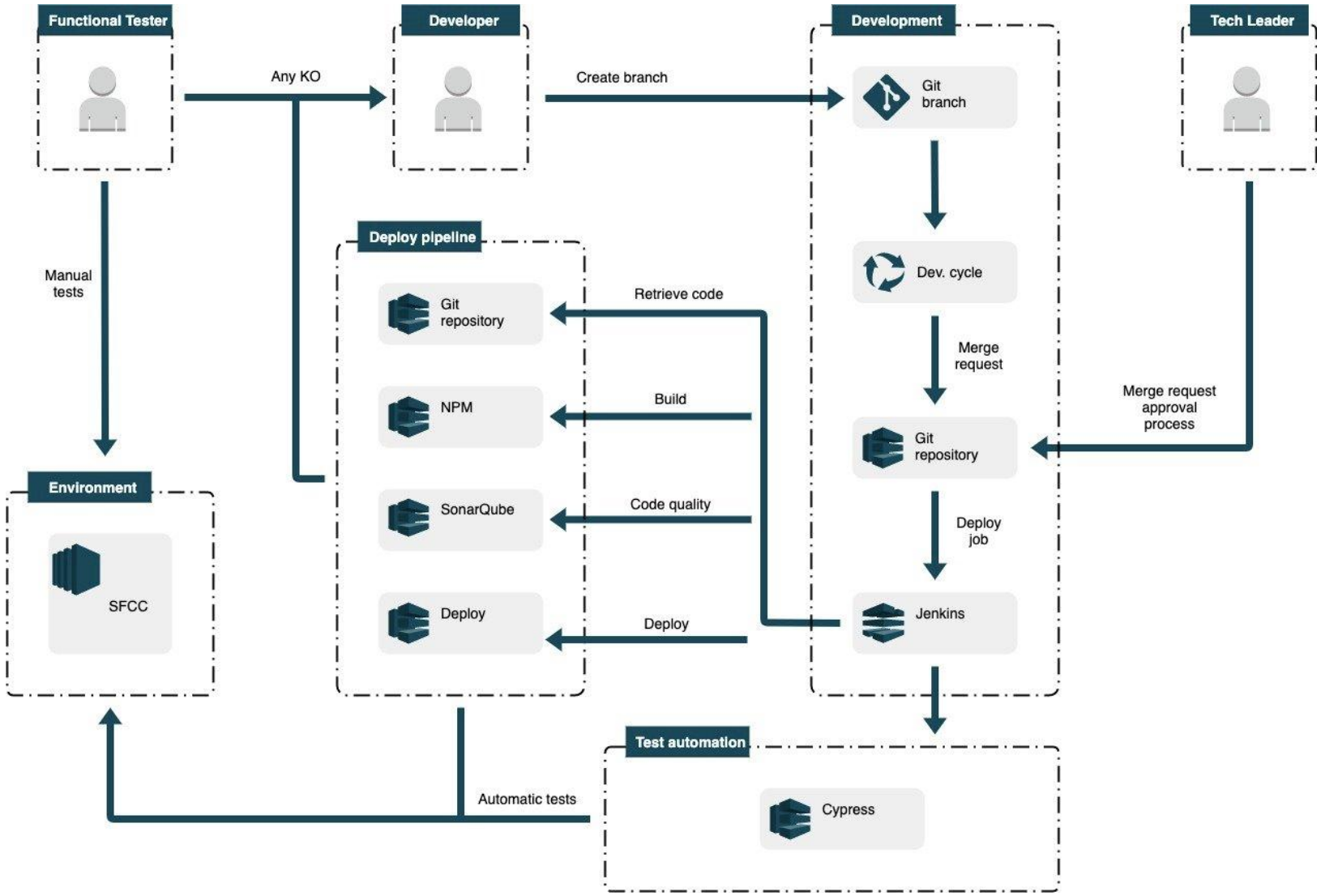
Ca' Foscari University of Venice

Real case: QA -process



Ca' Foscari University of Venice

Real case: QA -process



QA - benefits

- Methodology, processes and procedures to
  - Ensure proper quality of software
  - Share informations
  - Monitor the processes
  - Review and audit software products
  - Ensure that software meets standards
  - Ensure software match with business requirements
  - Improve entire software lifecycle

- Key benefits
  - Less rework
  - Less time for final UAT
  - Automatic regression test for enhancement
  - Reports about test results
  - More detailed documentation
  - Higher quality of software release

- Team benefits
  - Less stress
  - Easy maintenance of developments
  - More team interchangeability
  - Less effort to do the same things
  - Standardization of processes

Ca' Foscari University of Venice

The importance of a great context

- Process and tools are mandatory for a good QA
- But as said before: QA is a state of mind
- You need to create the right environment in order to maximise the results
- All team members have to
  - know and agree on the purpose
  - know the goals
  - know the processes and rules
- At the end, the differences, will be provided by a positive mood and a good feeling through team's member
- Each team's member have to trust on processes and results provided by other team's members